Go Huge or Go Home: Flexible Hugepage Swapping for Cloud VMs

[Research]

Anonymous Author(s)

ABSTRACT

Memory dominates the cost of cloud data centers, yet VMs in public clouds routinely sit on memory they never touch. Reclaiming this cold memory enables overcommitment, but today's systems lean on general-purpose OS swap paths that were never designed for virtualized workloads: they miss reclamation opportunities and force operators to maintain invasive kernel patches.

This paper makes the case for large-granularity swapping in VMs. We characterize when 2MB swapping pays off, then build Uswap around that regime. Uswap is a userspace memory-management framework that swaps VMs at hugepage granularity and treats them as opaque, requiring no guest cooperation. Two extension points keep the kernel modifications out of the loop: a policy API lets reclaimers, prefetchers, and multi-VM balancers be written and replaced in userspace, and a backend-agnostic storage interface supports diverse swap targets. Uswap integrates cleanly with Linux/KVM and rebalances memory across co-tenant VMs.

We evaluate Uswap against Linux kernel swapping across microbenchmarks and eight cloud workloads. Under proactive reclamation, Uswap holds on average resident memory at 61% of each VM's working set while preserving 98% of baseline performance, beating the Linux kernel on both. Under thrashing conditions where available memory falls below the required working set, Uswap retains up to 32 percentage points more of baseline performance than the Linux kernel, in both single-VM and multi-VM setups.

1 INTRODUCTION

Memory-intensive applications [1, 19, 61, 72] are an integral component of modern online services. In addition, memory has become the most expensive and sought-after resource in the cloud [53]. Effective utilization of this resource is therefore a first-order cost concern.

Users typically provision Virtual Machine (VM) memory to accommodate peak loads, deploying large cloud instance types to avoid running out of resources. As a result, VMs often under-utilize their memory, contributing to memory waste at the host level. At Google data centers, approximately 30% of server memory is not accessed for minutes and is considered cold [64], and across more than one million Azure VMs, over half exhibit a lifetime mean memory utilization range below 10% [62]. This cold memory can be reclaimed and repurposed.

To attain flexibility in memory consumption, operating systems (OS) commonly employ swapping [9, 16, 69], which conserves memory by paging cold memory to cheaper, slower storage media (e.g., disks or remote memory). Applying swapping to public-cloud VMs raises three challenges that off-the-shelf OS mechanisms do not address: guest opacity, page granularity, and the operational cost of kernel-side memory management.

First, many existing mechanisms are guest-cooperative, i.e., involving explicit coordination between guest kernel and the hypervisor. Mechanisms such as *ballooning* [56], *virtio-mem* [30], and *HyperAlloc* [76] require guest kernel modification. This poses a challenge in cloud environments that support *non-cooperative* VMs, where guests are treated as black boxes — the hypervisor lacks direct access to guest virtual addresses and cannot rely on guest cooperation. Thus, being **guest-transparent** is essential for a cloud memory reclamation system.

Cloud providers compete on tenant performance, and one technique hypervisors employ is provisioning VMs with hugepages (e.g., 2MB) to reduce Translation Lookaside Buffer (TLB) pressure and shorten page walks [2, 12]. This creates the second challenge: **preserving hugepage backing under reclamation**. Operating systems do not support hugepage swapping, and falling back to 4KB forfeits the benefits hugepages were chosen for, but swapping at 2MB pays a steeper per-fault cost. Prior measurements show 20–55% of 2MB pages remain idle for tens of seconds [2], so the opportunity at hugepage granularity is real if the trade-off can be managed. §3 analyses this trade-off and identifies the regime in which 2MB swapping pays off for cloud VMs.

The third challenge is the **operational cost of kernel-side memory management**. Custom kernel patches are expensive for cloud operators to maintain: they have to be ported across kernel versions, audited for security, and tested on every host type [57]. The same maintenance burden makes it costly to iterate on policies as cloud infrastructure evolves — new swap targets such as RDMA-attached memory [27] and CXL pools [41] appear, and new reclamation [38, 62] and prefetching [48] strategies are an active research area. The cost is amplified by the fact that cloud hosts already rely on userspace frameworks such as Open vSwitch (OVS) [60] and SPDK vhost [32] for networking and storage, which bypass the kernel for performance and are inherently difficult for a kernel-resident memory manager to coordinate with. Userspace policies that can be developed and replaced independently of the host kernel are therefore both a research enabler and an operational requirement.

We address these challenges with *Uswap*, a workload-agnostic userspace swap system. Uswap's per-VM reclaimer proactively infers each VM's working-set size (WSS) from EPT access bits and trims to it, freeing memory the host can repurpose to pack additional VMs onto the same hardware. Crucially, Uswap proactively reclaims at 2 MB granularity: the resident working set retains TLB and page-walk benefits (§3.1), and reclamation pays a lower scanning cost than a 4 KB system would (§3.3). Under slight pressure where reclamation extends just below the WSS, locality-friendly workloads stay in the region where 2 MB matches or outperforms 4 KB (§3.2). VMs change phase, however, and reclaimed pages must be swapped back in; prefetch policies cut swap-in latency by loading pages ahead of the fault. When several co-tenants phase-change

at once, aggregate demand can momentarily exceed physical memory; the Uswap balancer absorbs these transients by redistributing memory across VMs using per-VM page-fault and WSS signals. Sustained pressure is handled by a datacenter control plane, assumed to exist, through migration or eviction; without one, Uswap remains functional but cannot relieve host-wide pressure beyond the balancer's local action.

Uswap runs all swap decisions and I/O in userspace, separating policy from mechanism so that reclaimers, prefetchers, and balancers can be developed and replaced without kernel changes, with optional VM introspection available to policies that benefit from guest context. It integrates with KVM [7], OVS, and SPDK-based disk emulation through minimal host-kernel patches and no guest modifications, backed by HugeTLB-allocated VM memory. The storage backend is designed to be swap-target agnostic; this paper focuses on NVMe SSDs and extends to disk arrays via SPDK RAID, using zero-copy kernel-bypass I/O to keep per-fault cost low. Uswap is designed to enable memory-overcommit instance types analogous to burstable CPU instances [5, 25, 50], where tenants accept occasional swap-in latency for cheaper memory.

We summarize our contributions below.

- An examination of the trade-offs between performance, data granularity in VMs when utilizing 2MB versus 4KB pages with swapping mechanisms (§3).
- The design (§4) and implementation (§5) of Uswap, a userspace memory overcommit system that infers per-VM WSS to reclaim cold VM memory, with replaceable reclaimer, prefetcher, and multi-VM balancer policies for managing memory.
- We evaluate Uswap (§6) on microbenchmarks and eight cloud workloads against non-swapping and kernel-swap baselines. Under proactive reclamation, Uswap holds on average resident memory at 61% of each VM's WSS while preserving 98% of baseline performance, beating the kernel on both. Under hard memory limits, Uswap retains 85% of baseline performance at 70% of a single VM's WSS, and 89% of baseline under multi-VM overcommit at 60% of aggregate WSS, both outperforming the kernel. Custom policies and storage backends further demonstrate Uswap's flexibility.

2 BACKGROUND

Cloud. Modern datacenters provide virtualized compute, memory, storage, and networking through hypervisors that isolate workloads in VMs, supporting elasticity and fault tolerance via migration, resizing, and replication. To accelerate I/O, hypervisors leverage userspace frameworks such as SPDK and OVS, which bypass the kernel to provide high-performance, zero-copy access to storage and networking devices.

Nested Paging. Hypervisors use hardware-assisted nested paging: the guest manages GVA → GPA translation via its page table base register (PTBR), while the Memory Management Unit performs a second-level GPA → HPA translation through a hypervisor-maintained Extended Page Table (EPT [33]). Each guest page is also accessible via a host-virtual address (HVA) in the VMM's address space. EPT entries carry access and dirty bits that the hypervisor can read to infer which guest memory has been touched.

Hugepages. OSs commonly use 4KB pages as the default granularity, a choice that helps reduce swapping latency and limit memory fragmentation. However, 4KB pages increase TLB pressure and lengthen page table walks, which can degrade performance. To mitigate these costs, hugepages are widely used to reduce TLB misses and shorten page table walks [3, 23, 58], improving overall efficiency [47]. Linux supports 2MB pages through Transparent Huge Pages (THP) [55] and HugeTLB [20]; THP requires no application changes, whereas HugeTLB must be allocated explicitly. Unlike THP pages, which may be split back into 4KB pages by the kernel, HugeTLB pages remain intact, providing stable hugepage backing. This reliability makes HugeTLB attractive for virtualization hosts that benefit from consistent large-page mappings.

Linux Swapping and Control Groups. HugeTLB swapping is not supported in Linux, and a 2018 proposal to add strict 2MB swapping was never merged into mainline [17]. Linux can swap THP-backed memory, attempting to reclaim full 2MB pages before faulting pages back in at 4KB granularity. Over time, this fragments THP-backed memory into 4KB pages, reducing hugepage coverage (Fig. 1). The figure reports the average hugepage coverage across four cloud workloads under memory pressure (§6.6), showing that THP progressively approaches a 4KB swapping system.

Recent Linux kernels implement Multi-Gen LRU (MGLRU) [15], which tracks page generations through periodic scans and reclaims older ones under memory pressure [18]. Proactive reclamation can be driven from userspace via cgroups and the MGLRU interface [68, 75], but such approaches are unsuitable for opaque VMs: they lack visibility into key metrics (e.g., page cache usage) and are sensitive to VM page scrambling [59]. We nevertheless implement an MGLRU-based baseline for comparison (§6).

Resource Reclamation System Calls. The `madvise` [44] system call allows processes to provide the kernel with hints about the intended use of a memory region. In particular, `MADVISE_DONTNEED` informs the kernel that a region is no longer needed, allowing it to reclaim the associated physical pages while preserving the virtual address range. Subsequent accesses to the region will trigger a page fault. Similarly, Linux supports hole punching through `FALLOC_FL_PUNCH_HOLE`, which reclaims the physical storage backing a file range without changing its logical size [46].

Userspace Page Fault Handling. Userfaultfd (UFFD) is a Linux kernel mechanism [43] that allows userspace processes to handle page faults on designated memory regions. When a fault occurs, the kernel suspends the faulting thread and delivers the event to a userspace handler via a file descriptor, including the faulting virtual address. For HugeTLB pages, the handler resolves faults via `UFFDIO_CONTINUE`, which reinstalls an existing page cache entry without copying, resuming the faulting thread.

3 ANALYSIS OF HUGEPAGE SWAPPING

In this section, we analyze the opportunities and challenges of hugepage swapping.

3.1 Quantifying the 2MB/4KB trade-off

In a swapping virtualization system, there is a trade-off between the advantages of 2MB-hugepages (faster page walk, larger TLB

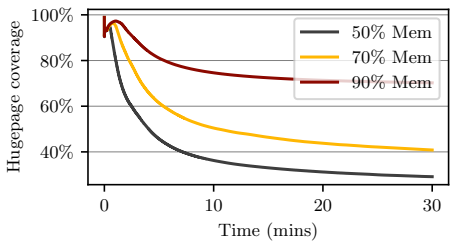


Figure 1: Hugepage coverage in THP over time swapping

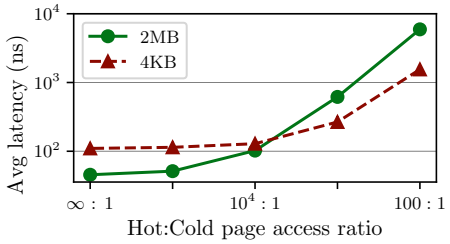


Figure 2: Avg. access latency hot vs. cold page access ratio

coverage, faster page table scans) and the advantages of 4KB swap operations (faster load latency). To quantify this trade-off, we run a microbenchmark in a VM accessing two memory regions: resident pages and swapped-out pages on an SSD. We gradually adjust the ratio of random memory accesses to the resident pages and swapped-out pages. We measure the average memory access latencies for both the *2MB* system and *4KB* system. The resident region is 100 GB and the swapped-out region is 28 GB on a 128 GB VM, the largest we could launch on our hardware. Both far exceed TLB reach for either page size, driving the miss ratio near its asymptotic maximum. Hence, this microbenchmark shows a lower bound for hugepages performance gains, as in practice hugepages also extend the TLB coverage.

Fig. 2 quantifies these effects: In the left part where only hot accesses happen (∞:1), the average access latency is 2.4× lower in the 2MB system. By contrast, when many cold accesses happen (such as 100:1), the speedup of the 4KB system is 3.0×. The *2MB/4KB break-even* is around 10^4:1 , above which the *2MB* system starts to be faster than the *4KB* system. This break-even point suggests that a 2MB swapping system can surpass a 4KB swapping system when enough of its working set remains in hot memory.

3.2 Real workload behavior

Whether the 2MB break-even of §3.1 holds for real workloads depends on access pattern (hotness fragmentation [10]) and available memory. We isolate the access-pattern effect by simulating Bélády's optimal cache [8] with single-vCPU runs and treat available memory as a parameter. We evaluate eight cloud workloads spanning web serving (Nginx), data-intensive processing (TPC-H, Redis, Elastic search), graph analytics (Graph500), HPC kernels (XSBench, Matmul), and stream processing (Kafka); the selection deliberately includes fragmented random access workloads (Redis, XSBench), where 2MB swapping performs poorly, to honestly characterize the boundaries of hugepage-based reclamation (Table 1).

We obtain 4KB and 2MB non-swapping baselines from two runs with different backing page sizes. In subsequent profiling runs, we collect each workload's page access history by scanning EPT access bits every 1 ms and computing Bélády-optimal swap counts at varying fractions of each workload's WSS. We multiply these counts by measured per-fault latencies (63 μs for 4KB, 551 μs for 2MB; Fig. 7) and add the result to the non-swapping baseline runtime, yielding projected runtimes normalized to baseline.

Fig. 3 shows three regimes. High-locality workloads (`matmul`, `kafka`, `TpcH`) consistently favor 2MB across all memory levels. Small-WSS, irregular workloads (`elastic`, `nginx`) see little 2MB benefit at full memory and cross over to 4KB once memory drops to 80% and 40% respectively. Large-WSS workloads split: `redis` and `XSBench` gain up to 10% from 2MB at full memory but lose to 4KB once memory falls below the WSS, while structured `Graph500` keeps the 2MB advantage down to roughly 50% of capacity. Overall, 2MB matches or exceeds 4KB for all workloads when memory is sufficient.

3.3 EPT scanning overheads

Access bits are flags in the page table that indicate whether a memory page has been recently read or written, allowing the system to track page usage. The process of scanning and zeroing the access bits to determine cold memory pages incurs system overheads [59]. To evaluate this overhead, we execute a read-only workload inside the VM that continuously accesses all of its memory. On a separate CPU core, we perform EPT scanning, reading, and, clearing of access bits from the VM's EPT, without flushing the TLB. We monitor scan time and performance impact on the running workload.

~~Unsurprisingly,~~ Fig. 4 shows that frequent EPT scanning increases overhead, which can be split into *direct* (CPU time spent scanning [78]) and *indirect* (workload slowdown due to partial walk cache flushes [33]). Scanning a 4GB VM with 4KB pages more often than every second saturates the CPU. In contrast, 2MB pages incur lower overhead due to fewer page table entries. The scan interval reflects a trade-off: shorter intervals improve access visibility but increase system cost. To fine-tune precision during runtime, swap policies must be able to regulate scan frequency. Additionally, employing 2MB pages helps minimize page table size and shortens scan duration.

4 DESIGN

The trade-off between 2MB and 4KB swapping creates a narrow "sweet spot": hugepages offer significantly lower scanning overhead and better TLB performance, but carry a steep fault penalty if memory pressure becomes too severe. To capture these benefits while respecting the cloud-specific mandates of guest transparency and operational maintainability, the system must be stable, proactive, and modular. USWAP achieves this by moving swap logic to userspace and enforcing a non-fragmenting hugepage path.

We first motivate the design decisions that flow from these principles, then describe the resulting architecture.

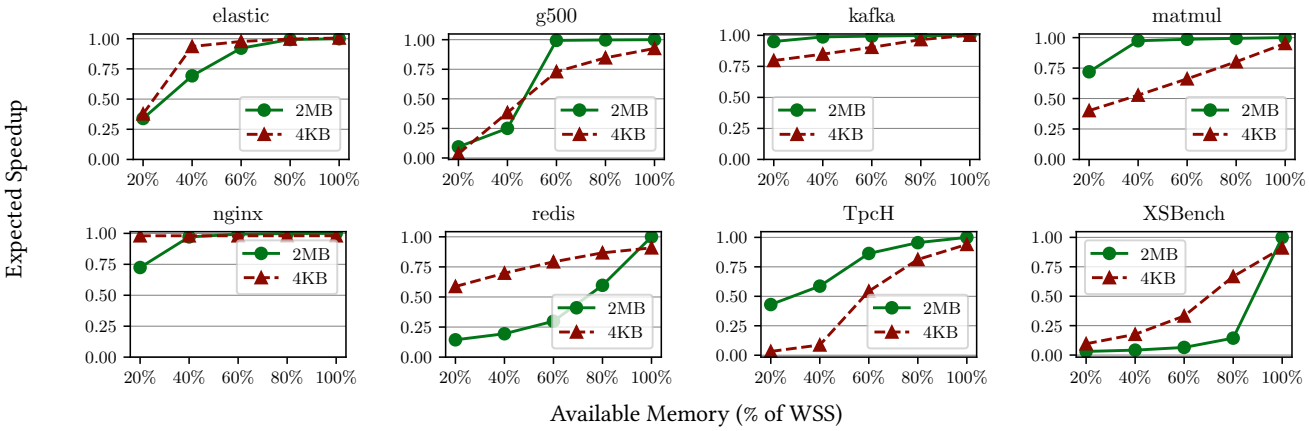
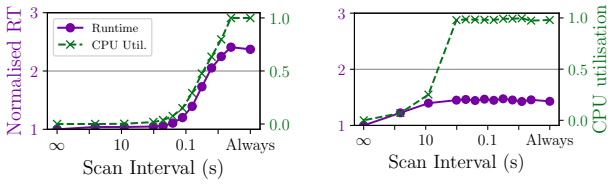


Figure 3: Bélády comparison between 2MB and 4KB



(a) 2MB pages **(b) 4KB pages**

Figure 4: Direct (*CPU%*) and indirect (runtime) costs of higher EPT scan frequency: 2MB vs 4KB pages. 4GB VM

4.1 Design rationale

HugeTLB over THP. §3 identifies clear break-even points between 4KB and 2MB swapping: hugepage swapping wins only when memory availability stays above these thresholds. THP is ill-suited to a strict 2MB swap path because the kernel reclaims THP-backed memory at 4KB granularity, progressively fragmenting hugepage coverage and degrading the system into a 4KB swap mechanism that forfeits the benefits established in §3. HugeTLB provides stable, non-degrading 2MB allocations natively, without requiring the kernel modifications a strict-THP swap path would demand.

The drawback is that HugeTLB performs poorly under random-access workloads or high overcommit; Uswap mitigates this by pairing HugeTLB with a reclaimer and a balancer that together aim to minimise the page fault count and keep VMs in the regime where hugepage swapping is beneficial, without requiring explicit knowledge of each opaque VM's break-even point. When memory pressure cannot be relieved within these constraints, we rely on the control plane to intervene.

Userspace over kernel. To reduce operational overhead on cloud providers, Uswap implements all swap decisions and I/O in userspace. This accelerates development of new policies, supports stateful policies, and integrates naturally with KVM, QEMU, and userspace I/O backends such as SPDK-vhost. It also decouples policy from mechanism: reclaimers, prefetchers, and balancers can be developed and replaced without kernel changes.

Guest-transparent over cooperative. Uswap treats VMs as black boxes and does not rely on cooperative mechanisms such as *virtio-balloon* [73] or *virtio-mem* [30], both of which require enablement or guest kernel modifications. Such modifications complicate deployment in cloud settings, where providers must supply custom images or tenants must modify their kernels. Cooperative approaches also still require a fallback for guests that do not cooperate or stop doing so, which is the role Uswap fills.

Optional introspection. Some policies, such as a next-page prefetcher, need to reason about guest virtual addresses, but the GVA→GPA mapping grows increasingly irregular over time [78]. To support such policies without modifying the guest, Uswap exposes an optional VM introspection interface that surfaces guest context at page fault time and provides best-effort GVA-to-HVA translation (§4.3). Policies that do not need this context pay no cost for it.

4.2 Architecture overview

Fig. 5 shows the architecture of Uswap. Three processes cooperate to manage VM memory: the Uswap daemon (UswapD), a per-VM memory manager (Uswap-MM), and a global storage backend. UswapD starts at boot and spawns, configures, and balances memory across VMs. Each Virtual Machine Monitor (VMM) process (QEMU) registers with UswapD ❶, specifying its memory configuration (e.g page size); UswapD selects appropriate parameters and launches a Uswap-MM ❷. The storage backend is a standalone application that serves I/O requests from multiple Uswap-MMs.

Uswap-MM hosts the Policy Engine and the components it coordinates. These fall into two classes. *Mechanisms* (§4.3) are mandatory components that implement demand paging and reclamation: the Uswapper, the UFFD Handler, the EPT Scanner, and the Policy Engine itself. *Policies* (§4.4) are optional components that decide *what* to swap and *when*: reclaimers, prefetchers, and other event-driven modules. Each Uswap-MM also exposes configuration knobs and runtime statistics that external services, such as the control plane or the UswapD balancer, use to monitor and adjust behaviour.

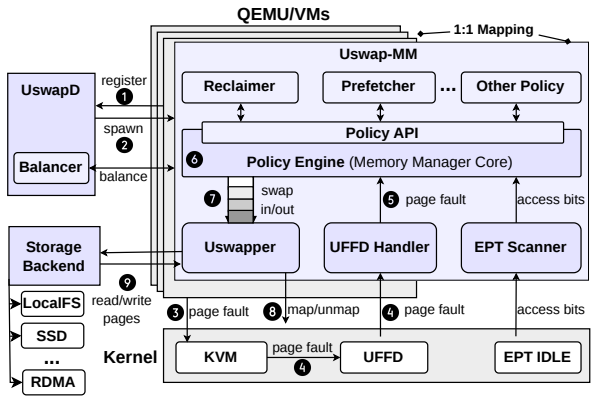


Figure 5: USWAP system overview

We illustrate the interaction between these components through two architectural walkthroughs, with steps numbered in Fig. 5.

Walkthrough I: Life of a page fault. When a VM accesses swapped-out data, it causes an EPT violation in the hypervisor ❸. The Linux memory subsystem sees the region is backed by userfaultfd [43] and forwards the event to userspace ❹. The UFFD Handler propagates it to the Policy Engine ❺, which ❻ checks the memory limit, enqueues a work item in the Uswapper queue, and notifies policies asynchronously. A Uswapper worker picks up the item from the queue ❼, contacts the storage backend via a shared-memory channel, and requests the page ❾. The storage backend reads the data from the storage device and populates the VM's memory directly; once complete, the Uswapper worker instructs the kernel to map the faulting page ❽ into the VM's address space and completes the UFFD event. The EPT violation is resolved and the VM resumes execution.

Walkthrough II: Reclaiming unused memory. The EPT Scanner periodically reads access bitmaps from the kernel module described in §5.2 and forwards them to subscribing policies. A reclaimer policy uses these bitmaps to identify cold pages and requests reclamation through the Policy API. The Policy Engine verifies the page is still resident and issues a swap-out request to Uswapper, which unmaps the page from all clients, asks the storage backend to persist it, and deallocates the host memory.

4.3 Mechanisms

The USWAP-MM hosts four always-active mechanisms that together provide demand paging and reclamation: the Uswapper, the UFFD Handler, the EPT Scanner, and the Policy Engine. Policies (§4.4) interact with these mechanisms exclusively through the Policy API and have no direct access to the kernel interfaces below.

Uswapper & queue. The Uswapper performs the actual page-in and page-out I/O, coordinated through a priority queue. It runs a pool of worker threads that dequeue requests (faults, reclaim, prefetch) from the Uswapper queue, drive the storage backend over a shared-memory channel, and complete UFFD events. The queue enforces request prioritisation (faults over prefetches) and respects memory constraints by ensuring sufficient eviction before new loads. Entry coalescing on insertion allows the queue to suppress redundant page-out requests if a fault arrives before an eviction is processed. To support third-party DMA clients (e.g., OVS via DPDK), the Uswapper also respects a lightweight page-locking mechanism, skipping locked pages during swap-out (§5.2). The Uswapper is the only component that maps swapped-in pages or deallocates host memory, ensuring policies never invoke kernel paths directly.

UFFD Handler. The UFFD Handler listens to userfaultfd [43] events for all VM-backed memory ranges. On a fault, it attaches optional guest context (e.g., PTBR, GVA) supplied via a dedicated kernel–userspace ring buffer (§5.2) and forwards the event to the Policy Engine. This introspection allows policies to reason about guest application state or make spatial predictions in virtual address space.

EPT Scanner. The EPT Scanner periodically reads access bits from the VM's Extended Page Tables, clears them, and produces an access bitmap that policies can subscribe to. The scan frequency is configurable, with the trade-off characterised in §3.3. The bitmap is the primary signal for cold-page detection; the scanner does not itself decide which pages to reclaim.

Policy Engine. The Policy Engine is the bridge between mechanisms and policies. It coordinates events (page faults, scan results) and policy-issued requests (reclaim, prefetch) by enqueuing work items into the Uswapper queue. It updates accounting against the configured memory limit, triggering forced reclamation (§4.4) if a request exceeds the budget. It also provides a best-effort GVA-to-HVA translation API (§5.2) for policies using guest context and enforces a correctness invariant: a policy-issued swap-in cannot restore content into an incorrect host page, assuming policies respect the Policy API.

4.4 Policies

Policies decide *what* to swap and *when*, while the mechanisms in §4.3 carry out the work. USWAP ships with default policies for reclamation and prefetching; both are replaceable. A policy subscribes to events (page faults, access bitmaps, periodic timers), reacts to them, and issues requests through the Policy API. Each policy runs in a dedicated thread with its own event queue so that a slow policy cannot block fault handling. The Policy Engine treats all policy requests as advisory: it admits them subject to the memory limit, drops them if the limit cannot be met (prefetches) or escalates them to forced reclamation (faults), and never lets a policy bypass the mechanism layer.

Reclaimers. A reclaimer subscribes to EPT-scan results and decides which resident pages ~~are cold enough~~ to evict. USWAP's default reclaimer is the *dt-reclaimer*, based on [38], which maintains per-page access-distance histograms and reclaims pages absent from the last N scans; N is chosen to bound the expected fault rate to a configurable target X over the next interval. The full algorithm and its parameter tuning are in §5.3. Other reclaimers can be plugged in by subscribing to the same events; §6.5 demonstrates

a phase-aware reclaimer that adjusts its scan interval based on page fault spikes, reducing the total number of scans.

Prefetchers. A prefetcher reacts to page fault events and predicts which pages are likely to be needed next, requesting them through the Policy API ahead of demand. Predictions in physical-address space are limited; richer prefetchers benefit from the optional VM introspection layer (§4.3) to reason in guest virtual address space. To illustrate the Policy API, we show below a next-page prefetcher that interprets the guest application's virtual address space, finds the next virtual page, and prefetches the corresponding physical page. The example assumes x86, where CR3 holds the page-table base.

```
void on_page_fault(page, cr3, gva) {
    if (!cr3 || !gva) {
        // Page fault has no associated CR3 or GVA info.
        // Don't prefetch.
        return;
    }
    next_gva = gva + page.size();
    next_hva = uswap.gva_to_hva(next_gva, cr3);
    if (!next_hva) {
        // GVA to HVA can fail, don't prefetch.
        return;
    }
    uswap.prefetch(next_hva);
}
```

The callback receives the page fault event augmented with the GVA and CR3 from the introspection layer. The policy increments the GVA by the page size, asks Uswap to translate it to an HVA, and issues a prefetch. Translation is required because the hypervisor operates on HVAs; the API is best-effort and may fail on stale guest page tables, in which case the policy skips the prefetch.

Forced reclamation. Forced reclamation is not a separate policy but a fallback the Policy Engine invokes when a page fault would push memory usage above the configured limit. It executes synchronously on the fault path, evicting just enough pages to admit the incoming one. The default implementation uses LRU over resident pages and is replaceable in the same way as the reclaimers above; the dt-reclaimer's accounting state can be reused if a workload-aware fallback is preferred.

4.5 Storage backend

The Storage Backend is a separate, global, userspace process that handles all swap I/O on behalf of Uswap-MMs. It receives requests from Uswapper workers over a shared-memory channel and exchanges data with one or more I/O devices, multiplexing requests across multiple Uswap-MMs. The interface is swap-target agnostic and accommodates page transfers to and from any block-addressable cold storage: NVMe SSDs, NVMe arrays, network-attached block devices, and RDMA-backed targets. This paper focuses on NVMe via SPDK, with implementation details in §5.4.

A single global backend is necessary because the kernel-bypass frameworks that make this performance possible, SPDK and DPDK, require exclusive device ownership in one process to provide zero-copy, poll-mode I/O. Consolidating swap I/O in one process also yields system-wide fairness across Uswap-MMs and bounds the trust surface to a single auditable component (§4.7).

4.6 Balancer

The balancer is a global component in UswapD that coordinates memory limits across co-tenant VMs. It complements per-VM reclaimers in two ways. First, when a VM's working set grows after a phase change and its reclaimer cannot recover memory locally, the balancer raises that VM's limit by taking capacity from co-tenants with slack. Second, when host-wide demand approaches physical memory, the balancer redistributes the host budget to minimise total thrashing rather than letting independent reclaimers compete. Sustained pressure beyond what redistribution absorbs falls to the control plane, not the balancer (see §4.1).

The balancer operates exclusively on telemetry exposed by each Uswap-MM: per-VM WSS estimates, page fault counts over a recent window, and the current memory limit. It does not interfere with the per-VM reclaimer's decisions inside a VM's allocated budget; it only adjusts the budget itself. This separation keeps each Uswap-MM self-contained, avoids global synchronisation on the page fault path, and lets the balancer run at a coarser cadence than the per-VM reclaimers.

The balancer's allocation policy is replaceable, like other Uswap policies. Uswap's default policy minimises a thrashing-weighted objective over per-VM working-set estimates and a fault-sensitivity signal that captures how badly each VM responds to its current limit; the control law and its tuning are described in §5.5. Alternative balancers can be plugged in without changing Uswap-MM by subscribing to the same telemetry.

4.7 Trust boundaries

Processes shared by multiple VMs (UswapD and the storage backend) are fault-tolerant and can be restarted without terminating VMs. Failure or compromise of a Uswap-MM instance affects only its VM; we therefore run one Uswap-MM per VM. Arbitrary code execution inside a Uswap-MM is analogous to compromising QEMU: both map the guest RAM backing store and steer paging for that tenant, while other VMs remain confined to separate Uswap-MM processes. The storage backend, by contrast, is fully trusted and can read data from all VMs, but its logic is minimal beyond multiplexing storage requests. While this simplicity makes the core logic amenable to formal verification for functional correctness, Uswap remains susceptible to timing side channels, a challenge inherent to all shared swap systems and exacerbated by the opaque internal scheduling of modern SSDs [35].

5 IMPLEMENTATION

In this section, we present our implementation of Uswap. We focus on Intel architecture but our design is generic, and we successfully ported it to ARM as well.

5.1 Linux integration

Fig. 6 shows Uswap's processes and their memory mappings. Uswap is built on Linux shared memory and UFFD. Applications such as QEMU and OVS use Uswap-lib to allocate Uswap-managed memory: Uswap-lib creates a memory-backed file, maps it into the caller's address space, registers the mapping with UFFD, and forwards the UFFD information to Uswap-MM. Both Uswap-MM

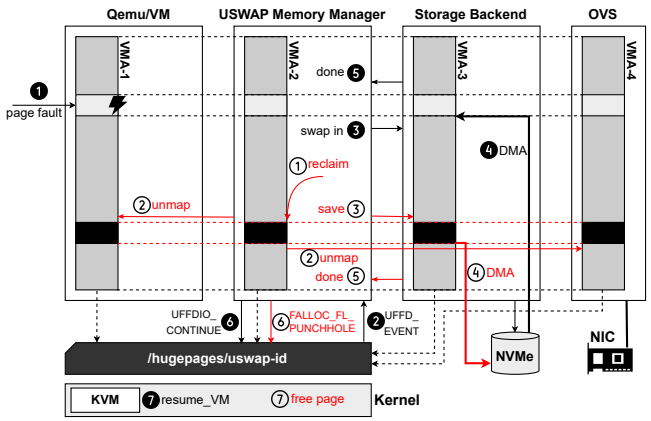


Figure 6: Swap-in (in black) and swap-out (in red) in Uswap using shared memory

and the storage backend map the same shared memory file into their own address spaces.

Swap-in path. When a client (e.g., QEMU or OVS) accesses memory that is not mapped in its address space, a page fault ❶ triggers a UFFD event at Uswap-MM ❷. The UFFD Handler forwards the event to the Policy Engine, which enqueues a request into the Uswapper queue. An Uswapper worker thread dequeues the request and contacts the storage backend ❸, which instructs the NVMe SSD to DMA the page's content directly into the VM's memory ❹. This is possible because the storage backend has the VM's memory mapped in its own address space. Once the content is populated ❺, the Uswapper worker issues a `UFFDIO_CONTINUE` ioctl ❻, resolving the page fault and resuming the client. To avoid the ~100 μs cost of zeroing a fresh 2MB page on the critical swap-in path, we maintain a zero-page pool refilled during idle time.

Swap-out path. When a reclaimer policy or the Policy Engine decides to swap out a page ①, an Uswapper worker executes the request. It first unmaps the page from each client ② via `MADV_DONTNEED`. Once the page is unmapped from all clients, the worker asks the storage backend to persist it ③, which DMAs the contents to the NVMe SSD ④ and signals completion ⑤. Finally, the Uswapper worker issues an `FALLOC_FL_PUNCH_HOLE` syscall ⑥ on the backing memory file, returning the memory to the kernel.

5.2 Mechanisms

This subsection describes how the mechanisms introduced in §4.3 are implemented: the EPT Scanner, the optional VM-introspection layer, and the page-locking interface for third-party DMA.

EPT scanner. Scanning the EPT is only possible from kernel space. We therefore ship a Linux kernel module derived from the Intel *memory-optimizer* [22], which exposes access-bitmaps of KVM-based VMs to userspace. The EPT Scanner inside Uswap-MM is the userspace counterpart: it aggregates policy-issued scan requests, drives the kernel module, and forwards the resulting bitmaps to subscribing policies on each scan tick.

To support VIRTIO [67] devices, the scanner must additionally inspect the page tables of host-side processes that DMA directly into VM memory, since those accesses do not register in the VM's EPT. For example, the page tables of OVS (which consumes network packets on behalf of guests) are scanned and the resulting access bits are merged with those from the VM's EPT before delivery to policies.

VM introspection: fault context. As described in §4.3, Uswap attaches guest context to each page-fault event. We add a kernel–userspace ring buffer between KVM and Uswap-MM to carry this metadata out-of-band, avoiding the cost of routing it through Linux's memory subsystem. KVM is modified to copy specific registers—the page table base register (PTBR, i.e., CR3 on x86), the instruction pointer (IP), and the faulting guest virtual address (GVA)—from the virtual machine control structure (VMCS) into the ring buffer on each EPT violation. When Uswap-MM later receives the corresponding UFFD event, it consumes the matching entry from the ring buffer and attaches this context to the event before delivering it to policies.

VM introspection: GVA-to-HVA translation. Policies that reason in guest virtual address space need to convert a predicted GVA into a host virtual address (HVA) before issuing a request through the Policy API. Uswap exposes this translation as an asynchronous API: a policy submits a target GVA together with the PTBR context (obtained via the fault-context ring buffer) it should be resolved against. Uswap forwards the request to the VMM (QEMU), which walks the guest page tables on a dedicated thread and returns the HVA. Unlike the existing `KVM_TRANSLATE` ioctl, the guest is not paused during the walk. The API is best-effort: it can fail if guest page tables are modified or if the GVA is unmapped, in which case policies (such as the prefetcher in §4.4) simply skip the request.

Page-locking for third-party DMA. Cloud hosts often deploy zero-copy I/O paths in which third-party userspace processes DMA directly into VM memory. OVS via DPDK is the canonical case: OVS maps guest buffers and instructs the NIC to read or write them in place. This is a hazard for any swap system, since a DMA in flight against a page that has been swapped out compromises integrity. Hardware solutions exist [40] but are not widely deployed in datacentres.

Uswap exposes a lightweight page-locking mechanism for such clients. Each client shares a page-lock bitmap with its Uswap-MM recording which pages are DMA-pinned. Locking is a two-step protocol: the client atomically sets the page's bit, then reads the page to force a swap-in if not resident. Once both steps complete, Uswap-MM will not swap the page out until the client clears the bit. Uswap-lib wraps the protocol with convenience APIs for typical DMA workflows.

Kernel footprint. Uswap's kernel-side modifications are deliberately small. Exposing existing memory-management helpers to the EPT-scanner module requires 5 lines of symbol exports; the scanner itself is an out-of-tree loadable module derived from the Intel memory-optimizer [22]. Propagating VMCS registers to userspace for the optional VM-introspection layer (§5.2) adds 156 lines in KVM. Deployments that do not use guest-context-aware policies

need only the 5-line symbol-export patch. The rest of Uswap runs in userspace and uses only mainline kernel features.

5.3 Default reclaimer

Uswap's default reclaimer, the *dt-reclaimer*, is the implementation of the proactive reclamation policy described in §4.4, based on the design in [38]. It subscribes to access bitmaps from the EPT scanner (§5.2) and decides which resident pages to evict on each scan tick.

The reclaimer scans the EPT at a configurable interval, defaulting to 2 *s* for 2MB and 60 *s* for 4KB configurations. These values trade off cold-page detection against the scanning overhead in §3.3; the larger 4KB interval reflects its higher per-scan cost. The reclaimer keeps a sliding history of access bitmaps and treats a page as cold if absent from the last *threshold* bitmaps.

The threshold is not fixed. The reclaimer maintains a per-page access-distance histogram, updated on each scan, that records how many scan intervals elapse between successive accesses to the page. Aggregating the histograms across all resident pages produces a distribution from which the reclaimer computes a *proposed threshold*, set so that no more than $X\%$ of the predicted working set is expected to fault during the next interval. X is the *target promotion rate* and controls the aggressiveness of the reclaimer: a higher X tolerates more page faults in exchange for reclaiming more memory.

5.4 Storage backend

Uswap's storage backend (§4.5) targets NVMe as its primary swap target, which is readily available in cloud hosts. When a new Uswap-MM is spawned, it establishes a communication channel with the storage backend, and the backend maps the VM's memory into its own address space (§5.1). Uswap-MM and the backend communicate via a lock-free queue: Uswapper threads insert I/O requests and block on a semaphore awaiting completion; the backend dequeues requests, processes them, and signals the semaphore.

The backend uses SPDK [32] for high-performance I/O. It continuously polls queues from all connected Uswap-MMs and programs the NVMe DMA engine for asynchronous data transfer. Zero-copy is possible for 2MB pages via direct DMA, but 4KB pages require an extra copy because SPDK does not support DMA into non-HugeTLB memory. To improve throughput and page-fault latency, the backend can stripe swap data across multiple NVMe devices using SPDK's RAID 0 support [66], enabling parallel I/O and faster 2MB page transfers.

Addressing the SPDK overhead. The CPU core that runs an SPDK application cannot be used for anything else, so dedicating a core to swap I/O alone is not cost-effective. Many cloud hosts already run SPDK vhost for disk emulation, however; Uswap exploits this by providing a storage-backend library that can be integrated into an existing SPDK vhost application, reusing its polling time and adding minimal CPU overhead. If SPDK vhost is not available, the backend can fall back to other I/O paths such as Linux filesystem syscalls; §6.1 characterises the performance difference. SPDK is a performance preference, not a hard dependency: the architectural benefits of Uswap are preserved even without it, only per-fault latency increases.

5.5 Balancer

The balancer (§4.6) is implemented in UswapD and adjusts per-VM memory limits based on telemetry from each Uswap-MM. For fault isolation, Uswap uses separate Uswap-MM processes per VM, which makes a global LRU impractical; instead, millisecond-scale telemetry sharing provides enough information for the balancer to act effectively (§6.6).

Fault-sensitivity signal. Workloads respond differently when their memory limit falls below the WSS (§3.2), but a precise WSS measurement requires expensive page-table scans that cannot run frequently [59]. To capture how badly a VM is suffering under its current limit without relying on WSS alone, we introduce the *pf_weight* signal. It is computed by a per-VM proportional-integral-derivative (PID) controller whose error term is the number of page faults observed over a fixed window W.

$$\Delta\mathrm{pf}_i(t) = \mathrm{pf}_i(t) - \mathrm{pf}_i(t-W) \tag{1}$$

$$\mathrm{u}_i(t) = \Delta\mathrm{pf}_i(t) + \int_{t-W}^{t} \mathrm{pf}_i(\tau) - \mathrm{pf}_i(t-W)\, d\tau + \frac{\Delta\mathrm{pf}_i(t)}{W} \tag{2}$$

$$\mathrm{pf\_weight}_i(t) = u_i(t) \times \mathrm{page\_size}_i \tag{3}$$

Sensitivity can only be observed *after* the limit is applied, which is why we use a feedback controller rather than a feed-forward estimate. When the limit comfortably exceeds the VM's WSS, the page-fault rate drops and *pf_weight* converges to zero.

Allocation policy. Given *pf_weight*, the balancer treats $\mathrm{wss}_i + \mathrm{pf\_weight}_i$ as a proxy for each VM's demand and minimises a thrashing-weighted objective across all VMs:

$$\min \left(\sum_{i=1}^{n} \left(\frac{\mathrm{wss}_i(t) + \mathrm{pf\_weight}_i(t)}{\mathbf{limit}_i(t)} \right)^2 \right) \tag{4}$$

subject to

$$\sum_{i=1}^{n} \mathbf{limit}_i(t) \le \mathrm{host\_mem\_total} \tag{5}$$

$$\mathrm{min\_mem} \le \mathbf{limit}_i(t) \le \mathrm{vm\_mem\_max}_i \tag{6}$$

Squaring the ratio in Eq. (4) amplifies the penalty on VMs experiencing heavy thrashing relative to those operating comfortably. The optimisation has a closed-form solution via Lagrange multipliers, which lets the balancer recompute limits as often as every 1 ms. We fix W at 2 seconds in our evaluation.

6 EVALUATION

In this section, we evaluate Uswap's performance and its flexibility. §6.1 compares the Uswap mechanism to Linux kernel swap on microbenchmarks. §6.2 shows that Uswap's WSS approximation tracks the known WSS of a synthetic workload. §3 established that 2MB swapping can be faster given a sufficiently large memory allowance; §6.3 shows that Uswap's proactive reclaimer stays within this regime on real workloads and quantifies the resulting savings. §6.4 investigates performance under forced reclamation, the regime in which the memory limit falls below the working set. Finally, we

Workload	Category	Configuration	Perf. Metric
TpcH [71]	DB	SF1 MariaDB in-memory	$\texttt{runtime}^{-1}$
XSBench [70]	HPC	450M lookups, event mode	$\texttt{runtime}^{-1}$
elastic [21]	Search	Rally, 27 tracks	Throughput
g500 [52]	Graph	Scale 27, 16vCPU/128GB	$\texttt{runtime}^{-1}$
kafka [37]	Streaming	perf-test	Throughput
matmul	HPC	20480^2 DGEMM, 4 vCPUs	$\texttt{runtime}^{-1}$
nginx [54]	Web	30k 4KB + 5k 1MB files	Throughput
redis [61]	KV-store	12GB dataset, memtier	Throughput

Table 1: Evaluated workloads

Name	Page	Backend	Policy	Scan
Baseline	2MB	none	none	n/a
Kernel	THP	kernel swap	MGLRU	dyn.
Uswap 2M	2MB	SPDK	dt	2 s
Uswap 4K	4KB	SPDK	dt	60 s
UswapB	2MB	block (O_DIRECT)	dt	2 s
UswapS	2MB	SPDK	dt + stride	2 s
Uswap Dyn	2MB	SPDK	phase-aware	dyn.

Table 2: Configurations evaluated

demonstrate the flexibility of Uswap through three custom policies: a stride prefetcher (§6.4), a dynamic reclaimer (§6.5), and a multi-VM balancer (§6.6).

Machine setup. All evaluations are conducted on a server with eight Intel® Xeon® CPU Max 9468 processors, each with 12 cores. Power-saving features, hyper-threading, and high-bandwidth memory are disabled in the BIOS. The system is configured with 256GB of DRAM, distributed as 32GB per NUMA node. We use the custom Linux kernel 6.13 described in §5 as our hypervisor. The swap target is four dedicated server-grade 3.5 TB SOLIDIGM® D7-P5520 [65] SSDs, each connected over PCIe Gen4.

Workloads. Table 1 summarizes the representative cloud workloads used throughout §3.2 and the evaluation. The workloads span DB, graph analytics, HPC, key-value serving, search, web serving, and streaming applications, covering diverse locality and working-set behaviors. Client/server workloads (Nginx, Redis, and Kafka) use a dedicated load-generator VM; we report only the memory utilization of the VM under test.

VM setup. Our default VM is configured with 4 vCPUs and 16 GB of RAM. QEMU vCPUs are individually pinned to physical cores on the same NUMA node as the SSDs. In multi-VM deployments (§6.6), the Kernel baseline places all VMs in a single cgroup with a fixed total memory limit, whereas Uswap enforces the host memory budget through its balancer, which redistributes memory across VMs dynamically.

Configurations evaluated. Table 2 summarizes the swap configurations compared in this section. *Baseline* is a non-swapping 2MB HugeTLB setup and serves as the normalization point for all relative results. *Kernel* is native Linux swapping with default readahead, asynchronous page faults [24], and THP enabled, driven proactively through MGLRU. *Uswap 2M* and *Uswap 4K* differ only in backing page size and use the default *dt* access-distance reclaimer (§5.3). *UswapB* is Uswap 2M with its SPDK backend replaced by a raw block device accessed via `O_DIRECT`, isolating the contribution of SPDK (§6.1). Two further configurations introduced later as case studies of policy flexibility are *UswapS*, which adds a stride prefetcher (§6.4), and *Uswap Dyn*, a dynamic reclaimer (§6.5).

Kernel proactive reclamation. For the *Kernel* configuration we drive Linux swapping proactively through cgroups and MGLRU [15]. Each QEMU process is placed in its own cgroup. MGLRU performs its periodic scan, builds page generations, and adjusts its scanning frequency based on observed memory access behavior. Every 1 second we check whether a new generation has been introduced and instruct the kernel to reclaim all generations up to `max_gen_nr` − 2, the most recent generation MGLRU permits for reclamation, by writing to the sysfs `lru_gen` file.

This is the most aggressive reclamation configuration MGLRU supports without application changes, and it represents the upper bound of what the mainline Linux kernel can achieve on opaque VMs. We deliberately compare against this upper bound: a less aggressive configuration would improve performance retention but reduce memory savings, leaving the kernel strictly dominated on memory recovery. As shown in §6.3, even this configuration reclaims less memory than Uswap on most workloads while incurring higher performance overhead.

Comparing memory saved. To compare memory consumption across two runs of different durations, we divide the faster runtime into 5 s buckets and align these buckets with the slower run using benchmark-specific synchronization points (e.g., end of load phase, end of warmup). The slower run's buckets may therefore exceed 5 s in wall time. We then average the relative memory usage across buckets. This alignment is necessary because raw time-averaged memory usage would unfairly penalize a slower run that holds the same working set for longer; bucket alignment compares the two systems at equivalent points in the workload's progress rather than at equivalent wall-clock times.

6.1 Uswap mechanism performance

Uswap performs as much work as possible in userspace, so we evaluate how the userspace path compares against an in-kernel baseline. Since page-in latency dominates swap performance, we run a microbenchmark inside a VM that initializes a memory region, triggers a full swap-out, and then drives random page-aligned accesses. We compare the four configurations in Table 2: *Kernel*, Uswap 4K, Uswap 2M, and UswapB.

To isolate mechanism cost from policy effects, we disable prefetching, read-ahead, and asynchronous page faults for both Uswap and the kernel; these are evaluated separately in Sections 6.3 and 6.4. We extend the evaluation to 1, 2, and 4 SSD arrays. Uswap 2M and Uswap 4K use SPDK RAID-0; Kernel and UswapB use `mdadm` [45]. Stride sizes are 2 MB/1 MB/512 KB for 2 MB swaps on 1/2/4 disks, and 4 KB (matching the SSD block size) for 4 KB swaps.

Page fault latency. Fig. 7 breaks down page fault cost into software overhead (*VMEXIT*) and disk I/O. With a single disk, userspace swapping raises VMEXIT from 11 μs in Kernel to 26 μs for Uswap

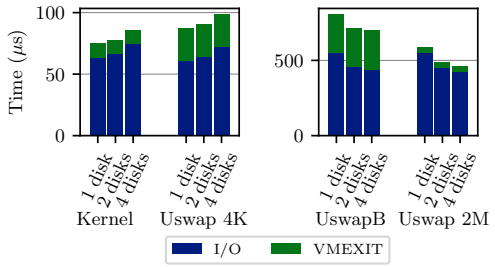


Figure 7: Page fault latency by backend and disk count

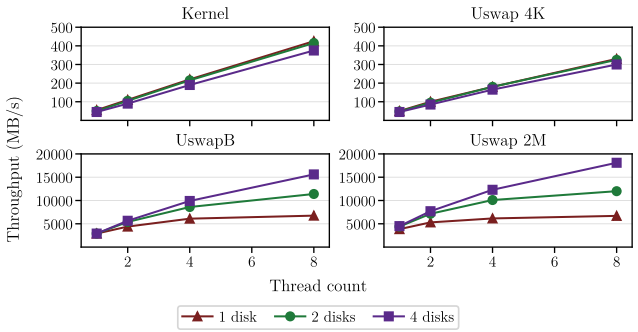


Figure 8: Throughput by backend and disk count

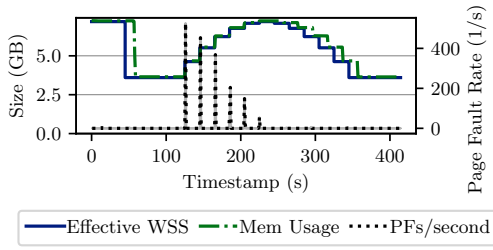


Figure 9: WSS, memory usage, and page fault rate over time

4K and Uswap 2M, but total latency grows by only 13 μs (16%) because I/O dominates: VMEXIT is just 6% of a Uswap 2M fault versus 18% in Kernel. ~~The absolute cost of a Uswap 2M fault is still 8.7× a Kernel fault, since it loads 512× more data.~~ UswapB, by contrast, incurs 253 μs of VMEXIT overhead despite differing from Uswap 2M only in storage backend; the Linux block layer, syscall crossings, and interrupt-driven completions that SPDK's kernel-bypass path avoids account for the gap. SPDK is therefore not merely a throughput optimization but what keeps userspace swap latency competitive with the in-kernel path.

VMEXIT stays constant as disks are added, so latency changes reflect I/O alone. For 4 KB configurations, latency rises from 63 μs/61 μs with one disk to 74 μs/72 μs with four (Kernel/Uswap 4K): each fault issues a single-block I/O confined to one stripe, so additional disks add RAID bookkeeping without I/O benefit. 2 MB pages instead benefit from striping, with Uswap 2M and UswapB latency dropping from 551 μs to 452 μs (2 disks, 20%) and 422 μs (4 disks, 30%) as their large contiguous reads expose SSD parallelism.

Throughput. Fig. 8 shows throughput, which scales as the latency analysis predicts. The 4 KB configurations top out at 421 MB/s (Kernel) and 330 MB/s (Uswap 4K) at 8 threads, with no gain from additional disks and a small regression at 4 disks consistent with the latency increase above. 2 MB configurations scale with both threads and disks: on a single disk, Uswap 2M starts at 3.5 GB/s versus UswapB's 2.5 GB/s (40% gap reflecting per-fault latency), and both saturate at 6.7 GB/s by 8 threads. At 4 disks/8 threads, Uswap 2M reaches 18.1 GB/s while UswapB reaches 15.5 GB/s; Uswap 2M saturates disk bandwidth with fewer vCPUs because faster faults are serviced more quickly.

Overall, Uswap 2M converts available storage bandwidth into swap throughput more efficiently than any other evaluated mechanism. Where SPDK vhost is unavailable, UswapB remains a functional fallback at reduced performance. For the rest of the paper we use Uswap 2M with SPDK.

6.2 Working set size estimation

We show that Uswap accurately estimates the effective working set by running a synthetic VM workload with a known, varying WSS and comparing it to Uswap 's reported memory use.

Fig. 9 demonstrates effective WSS values reported directly by the microbenchmark and the memory usage and page fault rate reported by Uswap-MM. The default Uswap reclaimer (§5.3) can precisely estimate the actual WSS and the memory consumption adjusts to this estimate in a timely manner.

6.3 Performance Retention and Memory Savings

We evaluate Uswap's default proactive reclaimer against MGLRU under matched conditions: each VM runs without an externally imposed memory limit, and reclamation is driven by the per-VM reclaimer alone. We compare Uswap 2M, Uswap 4K, and Linux kernel swapping with MGLRU (*Kernel* in Table 2) against the non-swapping 2MB *Baseline*, using the workloads from Table 1. Fig. 10 shows per-workload results: relative memory is the VM's memory usage normalized to the workload's total WSS (lower is better), and relative performance is measured against the Baseline.

Across the eight workloads, Uswap 2M slows workloads by 2% on average while using 61% of WSS in resident memory, with a worst-case drop of 4% on `nginx` and `redis`; it reclaims more memory than the Kernel on seven of eight workloads (the exception is `nginx`, by 2%). The Kernel, configured with its most aggressive proactive MGLRU setup, slows workloads by 13% at 81% resident memory. Two effects drive this cost. First, MGLRU periodically reclaims all available mature pages (generations up to `max_gen_nr-2`), which can be overly aggressive: g500 pays a 61% performance penalty for a 21% memory saving. Second, MGLRU sometimes fails to create new generations and misses opportunities; in `Kafka`, both Uswap configurations reclaim over 50% of memory while MGLRU leaves pages stuck in `max_gen_nr-1`. Uswap 4K behaves similarly to the Kernel overall (16% performance, 22% reclamation), consistent with §3: under proactive reclamation, THP-backed memory progressively degrades into 4KB pages (Fig. 1), so the two systems converge. A

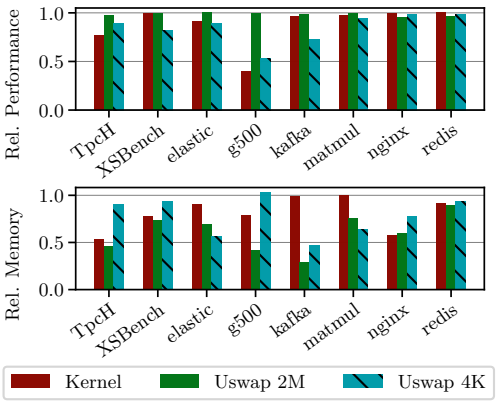


Figure 10: Relative memory savings (lower is better) and performance impact (higher is better) for three proactive reclaimers.

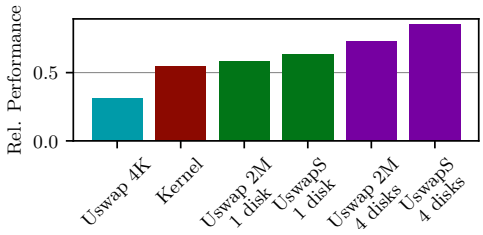


Figure 11: Matmul relative performance under a 70% memory limit, across configurations and disk counts.

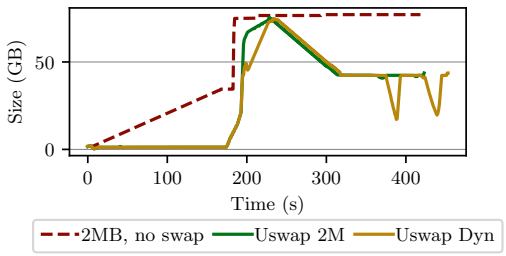


Figure 12: Memory usage of g500

contributing factor to Uswap 2M's higher savings is its 2 s EPT scan interval, versus Uswap 4K's 60 s and the Kernel's dynamic schedule; more frequent scanning is impractical at 4KB (§3.3).

6.4 Performance under Forced Reclamation

§6.3 evaluated Uswap when each VM's reclaimer was free to converge on its own working set. We now consider the opposite regime: the memory limit is set externally below the WSS, and Uswap-MM cannot relieve the pressure on its own. We compare Uswap against Linux swapping in this transient regime.

Workload behavior in this regime is primarily determined by memory access locality, as established in §3.2: high-locality workloads continue to benefit from 2MB swapping even with reduced memory, while low-locality workloads do not. We therefore evaluate two workloads at opposite ends of this spectrum: `matmul` (high locality) and `redis` (low locality, random access). For both, we estimate the workload's WSS and cap the VM's memory at 70% of that value. We compare four configurations from Table 2: Uswap 2M, Uswap 4K, the Kernel, and UswapS, which adds a stride-based prefetcher to Uswap 2M.

UswapS receives page-fault events and translates the faulting address into guest virtual address space using VM introspection (§5.2). If the last three faults are equally spaced, the next page in the stride is prefetched. The policy is under 200 lines of code, showing how the Policy API (§4) supports new workload-specific behaviors without mechanism-layer changes.

High-locality workload (matmul). Fig. 11 shows matmul performance under the 70% memory limit. Matmul's linear access pattern is well-suited to a stride prefetcher and retains performance well under forced reclamation thanks to its high locality. The Kernel retains 53% of baseline performance despite its own optimizations such as swap readahead and async page faults. On a single disk, Uswap 2M retains 58%, and stride prefetching (UswapS) lifts this to 63%. Scaling to four disks brings the full configuration to 85% of baseline.

Low-locality workload (redis). Redis at 70% memory limit is dominated by page-fault latency, since random access prevents both spatial prefetching and the locality assumptions that make 2MB swapping efficient. All evaluated 2MB configurations retain less than 10% of Baseline performance, and 4KB systems are a better fit for this access pattern (§3.2). More broadly, this is not a regime that a per-VM reclaimer can fix on its own: the appropriate response is to raise the VM's memory limit, either via the balancer (§6.6) when host capacity allows it, or via the control plane intervention.

6.5 Detecting Workload Phases

We demonstrate Uswap's adaptability by implementing the *dynamic* policy for phase-based workloads with changing working sets. When such a workload enters a new phase, page faults spike as swapped-out pages are touched. The policy detects this and enters *reclaim mode*, marking all pages old and adding them to an *old page set*. Uswap dynamic adjusts the scan interval according to detected working set changes: it decreases the interval to 1 second to remove active pages and avoid reclaiming recently accessed memory, reclaiming up to 2GB per scan cycle. Once the old set is drained, the system exits reclaim mode and returns to its default scan interval.

Using the g500 benchmark, we compare Uswap Dyn with a non-swapping baseline and default Uswap (Fig. 12). Uswap Dyn responds to the phase change within seconds, reaching its reclamation target as fast as the default reclaimer while scanning only every 60 s in normal operation. Averaged over the run, Uswap Dyn saves 2% more memory at a 6% performance cost. The case study illustrates the Policy API's expressiveness: workload-specific responsiveness is implementable in under 200 lines without mechanism changes.

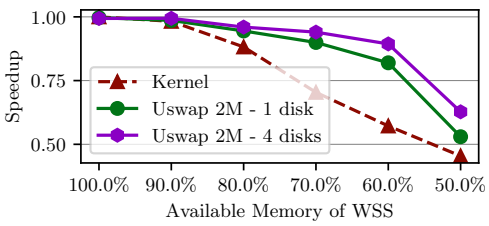


Figure 13: Average performance impact with multiple VMs as available memory is reduced

6.6 Multi-VM Memory Overcommitment

We next compare the balancer to Kernel on a host running multiple VMs with mixed workloads under varying degrees of memory overcommitment (Fig. 13). This captures the transient regime in which host memory is exhausted and the balancer must arbitrate it across VMs until the control plane intervenes. In the Kernel case, all VMs share a single cgroup (§6) and MGLRU decides which pages to swap out under pressure. The balancer instead sets per-VM limits from page fault counts and WSS estimates (§5.5), adjusting them automatically to minimize performance impact.

We launch 8 VMs (two per workload: redis, Tpc-H, g500 at scale 7, matmul), each looping its workload for 30 minutes. We report the geometric mean speedup over a non-swapping baseline, progressively reducing host memory below the aggregate WSS to force the balancers to redistribute.

The balancer steers memory toward latency-sensitive VMs based on their fault counts and WSS estimates. When such a VM falls below its WSS, the resulting faults raise its pf_weight and inflate its memory demand, forcing the balancer to expand its allocation at the expense of a more locality-friendly VM that has its limit set below its WSS but faulting less. On average, Uswap incurs total on average 100× fewer page faults than the kernel, even though its 2MB pages mean more data is swapped in per fault. The gap is widest at 60% memory availability, where the kernel retains only 57% of full-memory performance while Uswap retains 81% on a single disk and 89% with four disks.

7 RELATED WORK

Hardware-assisted page tracking. Uswap tracks idle pages within guests using EPT scanning, which records both read and write accesses across all pages. This gives complete page-level coverage at the cost of scanning overhead that grows with page-table size and scan frequency. Intel Page Modification Logging (PML) [31] lowers overhead by logging only write accesses, which is sufficient for VM live migration and snapshotting but misses read-only working sets relevant to reclamation. Processor event-based sampling (Intel PEBS) tracks hot pages by sampling, trading completeness for low overhead; MEMTIS [39] builds on PEBS for hot/cold tiering. PML and PEBS are well-suited to workloads where partial visibility is acceptable; Uswap uses EPT scanning because per-VM reclamation needs to identify cold pages comprehensively, including pages whose accesses PML and PEBS would miss.

Cold memory reclamation. Several works identify cold memory using various techniques and migrate it to byte-addressable, tiered, or far memory [10, 28, 36, 38, 41, 49, 75]. These approaches necessitate significant modifications to the Linux kernel and resemble a NUMA migration system more than a swapping mechanism.

The mainline Linux kernel recently introduced DAMON [59], a memory access monitoring subsystem that can also identify and reclaim cold memory. To mitigate the scanning overhead of §3.3, DAMON samples randomly within dynamically constructed "regions" of consecutive pages [78]. DAMON supports only virtual and physical address spaces, not EPT; while EPT support is feasible, region-based sampling is unlikely to be accurate under address-space scrambling, and prior work reports precision issues even without virtualization [39, 63]. A fair experimental comparison would require either porting DAMON to EPT or running it on guest-virtual address spaces, neither viable for opaque VMs and both incurring the kind of kernel-side maintenance burden Uswap is designed to avoid; we leave this to future work as EPT-aware access monitoring matures in mainline.

Swapping mechanisms. Prior work focuses on improving swap performance either by utilizing new NVMe interfaces [9], leveraging remote swapping over RDMA [4, 27], or performing prefetching in the swap subsystem [48]. Uswap is an extensible userspace swap system and framework, which allows the implementation of the prior approaches in userspace. vSwapper [6] enhances uncooperative VM swapping via page-zeroing detection and write deduplication. These optimizations are orthogonal to Uswap, but either require hypervisor kernel changes or assume co-located virtual disk and swap targets—an assumption often violated in cloud environments with network-attached block storage.

Page Fault Handling in Userspace. Userspace page fault handling—potentially by the faulting application—has been explored in microkernels [11, 29, 42]. Uswap adopts Linux's UFFD mechanism to benefit from its mature ecosystem. While Linux userspace fault handling remains an active research area (e.g., ExtMem [34] reduces latency), Uswap employs UFFD, as hugepage reliance amortizes its overhead. Moreover, UFFD is available in the mainline kernel.

Balancing. How to assign system resources to processes has long been studied [13, 14, 26, 51, 63, 74, 77, 79], Uswap can implement these in userspace.

8 CONCLUSION

We present Uswap, a transparent userspace swapping system for virtualization environments. Uswap supports strict 2MB swapping, integrates with commodity cloud software stacks, and works with non-cooperative VMs. Our evaluation shows that Uswap outperforms existing systems under conservative reclamation, delivering both higher performance and greater memory savings, and that its custom-policy support enables further workload-specific gains.

Cloud providers can leverage Uswap to increase VM density and improve memory utilization, while offering memory overcommit instance types where customers pay only for the memory they actively use, reducing costs for both providers and tenants.

REFERENCES

[1] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. 2016. TensorFlow: a system for Large-Scale machine learning. In *12th USENIX symposium on operating systems design and implementation (OSDI 16)*. 265–283.

[2] Neha Agarwal and Thomas F. Wenisch. 2017. Thermostat: Application-transparent Page Management for Two-tiered Main Memory. In *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems*. 631–644.

[3] Mohammad Agbarya, Idan Yaniv, and Dan Tsafrir. 2018. Memomania: From Huge to Huge-Huge Pages. In *Proceedings of the 11th ACM International Systems and Storage Conference*. 112–112.

[4] Emmanuel Amaro, Christopher Branner-Augmon, Zhihong Luo, Amy Ousterhout, Marcos K. Aguilera, Aurojit Panda, Sylvia Ratnasamy, and Scott Shenker. 2020. Can far memory improve job throughput?. In *Proceedings of the Fifteenth European Conference on Computer Systems (EuroSys)* (Heraklion, Greece). 1–16.

[5] Amazon Web Services. 2026. Burstable Performance Instances. https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/burstable-performance-instances.html. Accessed: 2026-05-20.

[6] Nadav Amit, Dan Tsafrir, and Assaf Schuster. 2014. VSwapper: a memory swapper for virtualized environments. *ACM SIGPLAN Notices* (2014), 349–366.

[7] Various Authors. 2025. Kernel Virtual Machine. https://www.linux-kvm.org/

[8] László Belády. 1966. A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal* (1966), 78–101.

[9] Shai Bergman, Niklas Cassel, Matias Bjørling, and Mark Silberstein. 2022. ZNSwap: un-Block your Swap. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)* (Carlsbad, CA). 1–18.

[10] Shai Bergman, Priyank Faldu, Boris Grot, Lluís Vilanova, and Mark Silberstein. 2022. Reconsidering os memory optimizations in the presence of disaggregated memory. In *Proceedings of the 2022 ACM SIGPLAN International Symposium on Memory Management*. 1–14.

[11] Brian N Bershad, Craig Chambers, Susan Eggers, Chris Maeda, Dylan McNamee, Przemyslaw Pardyak, Stefan Savage, and Emin Gün Sirer. 1994. SPIN: An extensible microkernel for application-specific operating system services. In *Proceedings of the 6th workshop on ACM SIGOPS European workshop: Matching operating systems to application needs*. 68–71.

[12] Ravi Bhargava, Benjamin Serebrin, Francesco Spadini, and Srilatha Manne. 2008. Accelerating Two-Dimensional Page Walks for Virtualized Systems. In *Proceedings of the 13th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 26–35.

[13] Jui-Hao Chiang, Tzi-Cker Chiueh, and Han-Lin Li. 2015. Memory Pressure Balancing on Virtualized Servers. In *Proceedings of the 2015 IEEE 21st International Conference on Embedded and Real-Time Computing Systems and Applications*. 70–79.

[14] Jui-hao Chiang, Tzi-Cker Chiueh, and Han-Lin Li. 2015. Memory Reclamation and Compression Using Accurate Working Set Size Estimation. In *Proceedings of the 2015 IEEE 8th International Conference on Cloud Computing*. 187–194.

[15] The Linux Kernel Development Community. 2023. Multi-Gen LRU. https://docs.kernel.org/admin-guide/mm/multigen_lru.html. Accessed: August 2025.

[16] Jonathan Corbet. [n. d.]. Reconsidering Swapping. https://lwn.net/Articles/690079/. Accessed: August 2025.

[17] Jonathan Corbet. 2018. The final step for huge-page swapping. https://lwn.net/Articles/758677. Accessed: December 2025.

[18] Jonathan Corbet. 2022. Merging the multi-generational LRU. https://lwn.net/Articles/894859/. Accessed: August 2025.

[19] Databricks. [n. d.]. Apache Spark: Unified engine for large-scale data analytics. https://spark.apache.org Accessed: August 2025.

[20] Linux Kernel Documentation. 2024. Hugetlb Page. https://www.kernel.org/doc/Documentation/vm/hugetlbpage.txt

[21] Elasticsearch. 2024. Elasticsearch Rally. https://esrally.readthedocs.io. Accessed: August 2025.

[22] Intel Fengguang Wu. 2019. *MemoryOptimizer - hot page accounting and migration daemon.* https://github.com/intel/memory-optimizer

[23] Alexander Fuerst, Stanko Novaković, Íñigo Goiri, Gohar Irfan Chaudhry, Prateek Sharma, Kapil Arya, Kevin Broas, Eugene Bak, Mehmet Iyigun, and Ricardo Bianchini. 2022. Memory-harvesting vms in cloud platforms. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 583–594.

[24] Red Hat Gleb Natapov. 2010. Asynchronous page faults. https://kvm-forum.qemu.org/2010/2010-forum-Async-page-faults.pdf Accessed: August 2025.

[25] Google Cloud. 2026. General-purpose machine family for Compute Engine. https://docs.cloud.google.com/compute/docs/general-purpose-machines#e2_machine_types. Accessed: 2026-05-20.

[26] Abel Gordon, Michael Hines, Dilma Da Silva, Muli Ben-Yehuda, Marcio Silva, and Gabriel Lizarraga. 2011. Ginkgo: Automated, application-driven memory overcommitment for cloud computing. *Proc. RESoLVE* (2011), 1–6.

[27] Juncheng Gu, Youngmoon Lee, Yiwen Zhang, Mosharaf Chowdhury, and Kang G. Shin. 2017. Efficient Memory Disaggregation with Infiniswap. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*. 649–667.

[28] Vishal Gupta, Min Lee, and Karsten Schwan. 2015. HeteroVisor: Exploiting Resource Heterogeneity to Enhance the Elasticity of Cloud Platforms. In *Proceedings of the 11th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments*. 79–92.

[29] Steven M Hand. 1999. Self-paging in the Nemesis Operating System. In *OSDI*. 73–86.

[30] David Hildenbrand and Martin Schulz. 2021. virtio-mem: paravirtualized memory hot(un)plug. In *Proceedings of the 17th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments*. 1–14.

[31] Intel Corporation. 2018. *Page-Modification Logging for Virtual-Machine Monitor.* Technical Report. Intel Corporation. https://www.intel.com/content/dam/www/public/us/en/documents/white-papers/page-modification-logging-vmm-white-paper.pdf

[32] Intel®. [n. d.]. *SPDK*. https://spdk.io Accessed: August 2025.

[33] Intel®. 2011. Intel® 64 and IA-32 architectures software developer's manual. *System programming Guide* (2011).

[34] Sepehr Jalalian, Shaurya Patel, Milad Rezaei Hajidehi, Margo Seltzer, and Alexandra Fedorova. 2024. ExtMem: Enabling Application-Aware Virtual Memory Management for Data-Intensive Applications. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. 397–408.

[35] Jonas Juffinger, Mathias Payer, Daniel Gruss, et al. 2025. Secret Spilling Drive: Leaking User Behavior through SSD Contention. In *Proceedings of the 32nd Network and Distributed System Security Symposium (NDSS)*.

[36] Sudarsun Kannan, Ada Gavrilovska, Vishal Gupta, and Karsten Schwan. 2017. HeteroOS: OS Design for Heterogeneous Memory Management in Datacenter. In *Proceedings of the 44th Annual International Symposium on Computer Architecture* (Toronto ON Canada). 521–534.

[37] Jay Kreps, Neha Narkhede, Jun Rao, et al. 2011. Kafka: A distributed messaging system for log processing. In *Proceedings of the NetDB*. 1–7.

[38] Andres Lagar-Cavilla, Junwhan Ahn, Suleiman Souhlal, Neha Agarwal, Radoslaw Burny, Shakeel Butt, Jichuan Chang, Ashwin Chaugule, Nan Deng, Junaid Shahid, Greg Thelen, Kamil Adam Yurtsever, Yu Zhao, and Parthasarathy Ranganathan. 2019. Software-Defined Far Memory in Warehouse-Scale Computers. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*. 317–330.

[39] Taehyung Lee, Sumit Kumar Monga, Changwoo Min, and Young Ik Eom. 2023. MEMTIS: Efficient Memory Tiering with Dynamic Page Classification and Page Size Determination. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 17–34.

[40] Ilya Lesokhin, Haggai Eran, Shachar Raindel, Guy Shapiro, Sagi Grimberg, Liran Liss, Muli Ben-Yehuda, Nadav Amit, and Dan Tsafrir. 2017. Page Fault Support for Network Controllers. In *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 449–466.

[41] Huaicheng Li, Daniel S. Berger, Lisa Hsu, Daniel Ernst, Pantea Zardoshti, Stanko Novakovic, Monish Shah, Samir Rajadnya, Scott Lee, Ishwar Agarwal, Mark D. Hill, Marcus Fontoura, and Ricardo Bianchini. 2023. Pond: CXL-Based Memory Pooling Systems for Cloud Platforms. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 574–587.

[42] Jochen Liedtke. 1995. On Microkernel Construction. In *Proceedings of the 15th ACM Symposium on Operating System Principles (SOSP-15)*.

[43] Linux man pages. 2020. USERFAULTFD(2): Linux Programmer's Manual. Accessed via man pages.

[44] Linux man pages. 2021. *madvise(2): Linux Programmer's Manual.* https://man7.org/linux/man-pages/man2/madvise.2.html

[45] Linux man pages. 2025. mdadm(8): Manage MD Devices aka Software RAID. https://linux.die.net/man/8/mdadm. Accessed: August 2025.

[46] Linux man-pages project. n.d.. fallocate(2): manipulate file space. https://man7.org/linux/man-pages/man2/fallocate.2.html Accessed: 2026-05-20.

[47] Aninda Manocha, Zi Yan, Esin Tureci, Juan L Aragón, David Nellans, and Margaret Martonosi. 2023. Architectural support for optimizing huge page selection within the OS. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. 1213–1226.

[48] Hasan Al Maruf and Mosharaf Chowdhury. 2020. Effectively Prefetching Remote Memory with Leap. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*. 843–857.

[49] Hasan Al Maruf, Hao Wang, Abhishek Dhanotia, Johannes Weiner, Niket Agarwal, Pallab Bhattacharya, Chris Petersen, Mosharaf Chowdhury, Shobhit Kanaujia, and Prakash Chauhan. 2023. TPP: Transparent Page Placement for CXL-Enabled Tiered-Memory. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (Vancouver BC Canada). ACM, 742–755.

[50] Microsoft. 2026. B family VM size series - Azure Virtual Machines. https://learn.microsoft.com/en-us/azure/virtual-machines/sizes/general-

purpose/b-family. Accessed: 2026-05-20.
[51] Debadatta Mishra, Purushottam Kulkarni, and Raju Rangaswami. 2019. Synergy: A Hypervisor Managed Holistic Caching System. *IEEE Transactions on Cloud Computing* (2019).
[52] Richard C Murphy, Kyle B Wheeler, Brian W Barrett, and James A Ang. 2010. Introducing the graph 500. *Cray Users Group (CUG)* (2010), 22.
[53] Onur Mutlu. 2013. Memory scaling: A systems architecture perspective. In *2013 5th IEEE International Memory Workshop*. 21–25.
[54] Nginx, Inc. 2025. *nginx*. https://nginx.org
[55] Alexandr Nikitin. 2017. Transparent Hugepages: measuring the performance impact. https://alexandrnikitin.github.io/blog/transparent-hugepages-measuring-the-performance-impact/
[56] OASIS Open. 2018. Virtio Specification v1.1. https://docs.oasis-open.org/virtio/virtio/v1.1/csprd01/virtio-v1.1-csprd01.pdf. Accessed: August 2025.
[57] Anel Orazgaliyeva. 2024. Working Towards Upstream-First. Kernel Recipes 2024, Paris, France. https://kernel-recipes.org/en/2024/working-towards-upstream-first/ Conference talk. Video: https://www.youtube.com/watch?v=BfRsAdAUo1I.
[58] Ashish Panwar, Sorav Bansal, and K Gopinath. 2019. Hawkeye: Efficient fine-grained os support for huge pages. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*. 347–360.
[59] SeongJae Park, Madhuparna Bhowmik, and Alexandru Uta. 2022. DAOS: Data Access-aware Operating System. In *Proceedings of the 31st International Symposium on High-Performance Parallel and Distributed Computing*. 4–15.
[60] Ben Pfaff, Justin Pettit, Teemu Koponen, Ethan Jackson, Andy Zhou, Jarno Rajahalme, Jesse Gross, Alex Wang, Joe Stringer, Pravin Shelar, et al. 2015. The design and implementation of open {vSwitch}. In *12th USENIX symposium on networked systems design and implementation (NSDI 15)*. 117–130.
[61] Redis. 2025. *Redis*. https://redis.io Accessed: August 2025.
[62] Benjamin Reidys, Pantea Zardoshti, Íñigo Goiri, Celine Irvene, Daniel S. Berger, Haoran Ma, Kapil Arya, Eli Cortez, Taylor Stark, Eugene Bak, Mehmet Iyigun, Stanko Novakovic, Lisa Hsu, Karel Trueba, Abhisek Pan, Chetan Bansal, Saravan Rajmohan, Jian Huang, and Ricardo Bianchini. 2025. Coach: Exploiting Temporal Patterns for All-Resource Oversubscription in Cloud Platforms. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 164–181.
[63] Jie Ren, Dong Xu, Junhee Ryu, Kwangsik Shin, Daewoo Kim, and Dong Li. 2024. MTM: Rethinking Memory Profiling and Migration for Multi-Tiered Large Memory. In *Proceedings of the Nineteenth European Conference on Computer Systems (Eurosys)*. 803–817.
[64] Zhenyuan Ruan, Malte Schwarzkopf, Marcos K Aguilera, and Adam Belay. 2020. AIFM:High-Performance,Application-Integrated far memory. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. 315–332.
[65] Solidigm. 2025. D7-P5520 PCIe 4.0 NVMe SSD for Data Centers. https://www.solidigm.com/products/data-center/d7/p5520.html Accessed: August 2025.
[66] Storage Performance Development Kit. 2025. *SPDK: Block Device User Guide*. SPDK Project. https://spdk.io/doc/bdev.html Accessed: August 2025.
[67] OASIS VIRTIO TC. 2022. Virtual I/O Device (VIRTIO) Version 1.2. *Committee Specification 01* (2022).
[68] Heo Tejun. 2015. Linux Kernel documentation: Control Group v2. https://docs.kernel.org/admin-guide/cgroup-v2.html
[69] The FreeBSD Project. [n. d.]. *FreeBSD Handbook*. Chapter Adding Swap Space. Accessed: August 2025.
[70] John R Tramm, Andrew R Siegel, Tanzima Islam, and Martin Schulz. 2014. XS-Bench - The Development and Verification of a Performance Abstraction for Monte Carlo Reactor Analysis. In *PHYSOR 2014 - The Role of Reactor Physics toward a Sustainable Future*. Kyoto. https://www.mcs.anl.gov/papers/P5064-0114.pdf
[71] Transaction Processing Performance Council. 2025. *TPC-H Benchmark*. http://www.tpc.org/tpch/
[72] Ververica. 2025. Apache Flink: Stateful Computations over Data Streams. https://flink.apache.org/. Accessed: August 2025.
[73] Carl A. Waldspurger. 2002. Memory resource management in VMware ESX server. (2002), 181–194.
[74] Zhigang Wang, Xiaolin Wang, Fang Hou, Yingwei Luo, and Zhenlin Wang. 2016. Dynamic Memory Balancing for Virtualization. *ACM Trans. Archit. Code Optim.* (2016).
[75] Johannes Weiner, Niket Agarwal, Dan Schatzberg, Leon Yang, Hao Wang, Blaise Sanouillet, Bikash Sharma, Tejun Heo, Mayank Jain, Chunqiang Tang, and Dimitrios Skarlatos. 2022. TMO: transparent memory offloading in datacenters. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 609–621. This introduces the PSI metric, facebook.
[76] Lars Wrenger, Kenny Albes, Marco Wurps, Christian Dietrich, and Daniel Lohmann. 2025. HyperAlloc: Efficient VM Memory De/Inflation via Hypervisor-Shared Page-Frame Allocators. In *Proceedings of the Twentieth European Conference on Computer Systems*. 702–719.
[77] Weiming Zhao and Zhenlin Wang. 2009. Dynamic memory balancing for virtual machines. In *Proceedings of the 2009 ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments*. 21–30.
[78] Yuhong Zhong, Daniel S Berger, Carl Waldspurger, Ishwar Agarwal, Rajat Agarwal, Frank Hady, Karthik Kumar, Mark D Hill, Mosharaf Chowdhury, and Asaf Cidon. 2024. Managing Memory Tiers with CXL in Virtualized Environments. In *Symposium on Operating Systems Design and Implementation*.
[79] Wenyu Zhou, Shoubao Yang, Jun Fang, Xianlong Niu, and Hu Song. 2010. VMCTune: A Load Balancing Scheme for Virtual Machine Cluster Using Dynamic Resource Allocation. In *2010 Ninth International Conference on Grid and Cloud Computing*. 81–86.